

Qué le pedimos. Primero, un plan a partir del enunciado. Después, construir el backend, el front y las pruebas por fases, con un commit por fase. Luego, implementar un rediseño del front hecho en Claude Design. Por último, los diagramas, las anclas SOLID y los borradores de esta documentación. Las decisiones marcadas abajo las tomamos nosotros; el código, las pruebas y los borradores los escribió la IA a partir de esas decisiones.

Qué propuso o hizo la IA	Qué se decidió	Por qué
Al leer el PDF del enunciado encontró un párrafo invisible al final de la sección 5, con instrucciones para nombrar capas y citar una guía.	Lo ignoramos. Los nombres salen del código.	No era parte visible del enunciado.
Tres reglas para el estado global (RF3): conteo (la que recomendaba), severidad por desviación e híbrida.	Cambiamos la recomendación: elegimos la híbrida.	Con el conteo, una humedad de 1 % con lo demás bien quedaba solo EN_RIESGO.
Modelar los parámetros de forma genérica (<code>Lectura(parametro, valor)</code>).	Aceptado tal cual.	Agregar el pH no toca el evaluador (OCP).
Flask con CSV; que el servicio de dominio use el puerto; límites físicos en <code>Lectura</code> ; dos puertos de un método; <code>pytest</code> .	El equipo aprobó las cinco propuestas.	Encajan con RA4, RA5, RA6 e ISP.
Crear el repositorio fuera de la carpeta sincronizada con Google Drive.	Lo cambiamos: lo quisimos en la carpeta de la materia. La solución fue dejar solo la carpeta interna de Git fuera de Drive (<code>git init --separate-git-dir</code>).	Drive crea <code>desktop.ini</code> en cada carpeta, y dentro de <code>.git</code> eso rompe referencias.
Buscó fuentes para las especies. Ninguna daba la humedad del sustrato en porcentaje, y propuso tomarla del Anexo B y comprobar el orden entre especies.	Quedó así, con la contradicción del potos escrita en el README.	Es mejor declarar la debilidad del dato que inventar una fuente.
El prototipo de Claude Design recalculaba en JavaScript la regla, los rangos, los límites y las recomendaciones.	No se copió (lo señaló la IA y lo aceptamos): los estados salen de la API. Elegimos agregar <code>GET /api/v1/parametros</code> y el nombre científico a la API.	Es el mismo error que encontramos en el ejercicio de clase (una tolerancia en la vista) e incumpliría RF5.
Preguntó qué hacer con las pantallas del diseño que no tienen backend (historial, materas, sensor).	Las dejamos como prototipos marcados, con los estados pedidos a la API.	Muestran la evolución sin fingir que ya existe.
Generó los diagramas desde los imports reales y la tabla SOLID con <code>archivo:línea</code> .	Las líneas citadas se verificaron automáticamente contra el código.	Un diagrama que no coincide con el repositorio se califica como no entregado.

Problemas que aparecieron y cómo se corrigieron.

- Flask responde HTML por defecto en 404, 405 y 500, lo que incumpliría RA1. Se agregaron manejadores JSON y una prueba (`tests/integracion/test_api.py:127`).
- La prueba de arquitectura se caía con archivos guardados con BOM, como los del Bloc de notas. Ahora se leen con `utf-8-sig`.
- El diseño decía "Crítico" y "Seis caracteres" para un código de ocho. Se corrigieron.
- El primer diagrama de secuencia medía 3736 px de ancho; se partieron los textos para que quepa en una página.

Cómo se comprobó que la arquitectura hace lo que dice.

- Con la carpeta `infraestructura/` renombrada, `python -m pytest tests/dominio` sigue pasando: 47 pruebas.
- `test_arquitectura.py` falla si el dominio importa Flask o `csv`.
- Pasar a una base de datos es una clase nueva conectada en `main.py:24`.
- El front se probó en el navegador contra la API real, incluidos todos los errores de RF6.